

Green and Heuristics-Based Consolidation Scheme for Data Center Cloud Applications

Alessandro Carrega^{1(✉)} and Matteo Repetto²

¹ DITEN – University of Genoa, Genoa, Italy
alessandro.carrega@unige.it

² CNIT – Research Unit of Genoa, Genoa, Italy
matteo.repetto@cnit.it

Abstract. The consolidation of resources is one of the most efficient strategies to reduce the power consumption in data centers. Various algorithms have been proposed in order to reduce the total number of required servers and network devices. The practice developed in response to the problem of server *sprawl*, a situation in which multiple, under-utilized servers (and/or network devices) take up more space and consume more resources than can be justified by their workload; with the effect to power off unused equipment. Generally, consolidation mechanisms consider different parameters related to the services neglecting the specific function of the Virtual Machines (VMs) in the application framework (e.g., core component, backup replica, member of a set of workers for load balancing).

In this work, we developed a new consolidation algorithm that takes into account the particular function of each VM with the aim to apply power saving mechanisms without compromising the desired service level. The results of the simulations show that it is possible to obtain significant values of energy saving. In particular, we show, with different heuristics, the optimal trade-off between service level and power efficiency achieved by the proposed model.

Keywords: Green · Energy-aware · Consolidation · Virtual machines · Cloud applications · Optimization · Heuristics

1 Introduction

The linear relationship between the power consumption and the current workload is the ideal objective to obtain an energy-efficient data centers. This goal can be realized with energy saving mechanisms, as, for example, frequency/voltage scaling, and *sleep* states [1].

The most efficient mechanism to reduce the power consumption is called *Sleeping*¹ [1]. [3] is the first work that introduced the concept of *resource consolidation* in order to maximize the effectiveness of this mechanism. In more

¹ Sleeping (i.e., ACPI S3 state [2]) consists of very low energy consumption states for electronic devices, where most functions are disabled but the system can resume to full operation in a very short time (typically under a few hundred milliseconds).

details, it is possible to sleep a significant amount of idle infrastructures (servers and network machines) grouping applications together on the smallest number of servers.

The first proposed strategies for consolidation only take into account the energy consumed by servers. These strategies model the power consumption of servers according to their current processing load (mainly in terms of CPU usage). Successive more advanced algorithms have been proposed considering network equipment as well as Heating, Ventilation and Air Conditioning (HVAC) systems [1]. In these cases, they also take into consideration the additional energy consumption generated by network traffic and the heat removal for various configurations of active servers in the particular data center installation. The central issue remains the need to migrate the applications between different servers, with interruptions of service time for short periods of time, that could be not acceptable for mission-critical applications [4, 5].

To reduce the power consumption without performance degradation, consolidation algorithms consider additional parameters as Quality of Service (QoS) metrics, Service Level Agreement (SLA), and network traffic patterns. Nevertheless, larger energy saving could be achieved by also taking into account the particular role of single Virtual Machines (VMs) into a complex service framework. In more details, this work is focused on elastic cloud applications [6], realized with different software components running in separate VMs. In this context, the workload variations make complex to find a right trade-off between performance, operational costs, and power consumption [7]. Indeed, for different reasons like backup, resilience, or performance (e.g., to serve sudden increases in requests), the service provider would need to run several idle replicas. However, this scheme would lead to inefficiency for infrastructure owners and higher operational costs for service providers, because of the VMs running idle most of the time.

In this work, we introduce a system model for energy efficiency management of cloud applications in data centers, which take into account both servers and network equipment. The main novelty of this mechanism is the specific classification of VMs into different groups, based on their particular role in the service framework they are part of. With such idea, the proposed algorithm enables infrastructure providers to selectively use aggressive energy saving mechanisms for servers that only run idle VMs, bringing them back to full operation in a matter of few hundred milliseconds (compared to a few minutes for typical VM provisioning). This opportunity allows achieving a better trade-off between energy efficiency and service level in real environments. We argue that the classification of VMs required by our algorithm could be easily inferred by specific information embedded by service developers into software templates, meta-models, and annotations, which are becoming common development paradigms for cloud applications [8–11].

The paper is organized as follows. Initially, Sect. 2 reviews related work. Section 3 describes our system model, while Sect. 4 describes the consolidation

algorithm. Section 5 shows results from our simulations. Finally, Sect. 6 gives our conclusions and plans for future work.

2 Related Work

Resource *consolidation* is one of the most effective ways to save energy in data centers. It implies gathering VMs on the minimum number of servers needed to fulfill their requirements (CPU, RAM, I/O bandwidth), in order to shut down the remaining infrastructure. The infrastructure under consideration always includes physical servers, and sometimes also encompasses network and HVAC equipment.

The consolidation problem has often been modeled as a modified *bin packing* problem, seeking the optimal operating point between energy and performance [12]. However, VM placement is not exactly equivalent to a three-dimensional (CPU, RAM, I/O) bin packing problem, and some anomalies could arise from this approach and similar ones in the literature [13]. Given the NP-completeness of the problem, most authors end up proposing heuristics to find a sub-optimal solution in acceptable time [12–15].

Several researchers have tackled the trade-off between traffic routing and power consumption in data center networks. Heller et al. [16] model the consolidation of network flows as a Multi-Commodity Flow Problem (MCFP) and a greedy bin packing. In other cases, VMs are explicitly placed to minimize the network power consumption. Shirayanagi et al. [17] target fault-tolerance and take into consideration placement of backup VMs. Wang et al. [18] cluster VMs according to their traffic patterns, and then place them on different racks with a classical k -cut algorithm; finally, the minimal number of network switches to carry the total traffic is computed, assuming a balancing algorithm is used (VLB/ECHP). The complexity and the large state of such systems are major computational problems; to this aim, hierarchical approaches have been proposed by Zhang et al. [19].

The problem of consolidation has also been modeled by jointly considering the power consumption of servers and network equipment. Kliazovich et al. [7] define a composite metric based on a hierarchical model that accounts for the power consumption of servers and network equipment, which is used to find the best balance between server utilization and queue size. Fang et al. [20] decompose the problem into three parts: (i) *traffic-aware grouping*, modeled as a Balanced Minimum k -cut Problem, (ii) *distance-aware VM-group to server-rack mapping*, modeled as a Quadratic Assignment Problem, and (iii) *power-aware inter-VM traffic flow routing*, modeled as a classic MCFP. Specific heuristics have been used to find a solution for each part.

3 System Model

We consider a data center made of physical servers, network devices, and links. Every application consists of one or more elementary software components, each deployed in a separate VM, which represents the granularity of our

model. VMs are created and destroyed according to the application life-cycle management logic, which is not relevant for the model and corresponds to an arrival/departure process. VMs are classified into different groups, depending on the power management mechanisms that can be applied without incurring in performance degradation.

3.1 Data Center Infrastructure Model

We model a data center as a weighted directed graph $\mathcal{G}(\mathbb{M}, \mathbb{E})$, where $\mathbb{M}$ and $\mathbb{E}$ are sets of nodes and edges, respectively. In the graph $\mathcal{G}$, a node $m \in \mathbb{M}$ is either a server (leaf) or network device (root or internal node); while the directed edge $e := (m_1, m_2) \in \mathbb{E}$ represents the network link from m_1 to m_2 (with $m_1, m_2 \in \mathbb{M}$) characterized by the bandwidth capacity C_{bw}^e (expressed in Mbps) and by the traffic utilization level $C_u^e \in [0, 1]$. Always referring to the edge $e := (m_1, m_2)$, we indicate the source and destination nodes with $C_{\text{src}}^e := m_1 \in \mathbb{M}$, and $C_{\text{dst}}^e := m_2 \in \mathbb{M}$, respectively. Besides, to simplify the equations in the following sections, we define the set of source and destination nodes $C_{\text{node}}^e := \{m \in \mathbb{M} \mid m = C_{\text{src}}^e \vee m = C_{\text{dst}}^e\}$, $\forall e \in \mathbb{E}$, too. Furthermore, we describe the opposite edge $\bar{e} \in \mathbb{E}$ of $e \in \mathbb{E}$ where $C_{\text{src}}^{\bar{e}} = C_{\text{dst}}^e \wedge C_{\text{dst}}^{\bar{e}} = C_{\text{src}}^e$. Considering the notion of edge, for each node $m \in \mathbb{M}$, we introduce the concept of neighborhood $\mathbb{M}_{\text{nb}}^m := \{\tilde{m} \in \mathbb{M} \mid m \in C_{\text{node}}^e \wedge \tilde{m} \in C_{\text{node}}^e \wedge e \in \mathbb{E}\} \subseteq \mathbb{M}$.

We denote with $\mathbb{S}$ and $\mathbb{N}$ the sets of servers and network devices, respectively (obviously $\mathbb{M} \equiv \mathbb{S} \cup \mathbb{N}$)². Considering that the number of ports can vary from one network device to another, we use σ_n to denote the number of ports into the network node $n \in \mathbb{N}$. The resources of a server $s \in \mathbb{S}$ are defined with a pair composed by the number of CPUs³ C_{cpu}^s and the quantity of RAM C_{ram}^s , expressed in MB.

For each pair of nodes $m_1, m_2 \in \mathbb{M}$, we define $\mathbb{P}$ as a set of network paths consisting in a walk from m_1 to m_2 with no repeated edges:

$$\mathbb{P} := \{(e_1, e_2, \dots, e_n) \mid C_{\text{src}}^{e_1} = m_1, C_{\text{dst}}^{e_1} = C_{\text{src}}^{e_2}, \dots, C_{\text{dst}}^{e_n} = m_2\} \quad (1)$$

To summarize, Table 1 shows all the notations introduced in this section (we use $\mathcal{Z}_{>0}$ and $\mathcal{R}_{>0}$ for the set of positive integer and real numbers, respectively).

3.2 VM Model

Data center users request VMs to build their applications; we do not need to explicitly take into account their belonging to specific applications since this aspect is already captured by the communication patterns. Hence, our model

² Based on the proposed node classification, we refer to $m \in \mathbb{M}$ with the term *device*, too.

³ For the sake of simplicity, in this paper, the term CPU refers to the minimum processing unit that can be set in a low-consumption state.

Table 1. Data center infrastructure model notation.

Parameter	Description
$C_{\text{cpu}}^s \in \mathcal{Z}_{>0}$	Number of CPUs provided by the server $s \in \mathbb{S}$
$C_{\text{ram}}^s \in \mathcal{R}_{>0}$	Amount of RAM (in MB) provided by the server $s \in \mathbb{S}$
$C_{\text{bw}}^e \in \mathcal{R}_{>0}$	Amount of network bandwidth (in Mbps) provided by the link $e \in \mathbb{E}$
$C_{\text{u}}^e \in [0, 1]$	Traffic utilization level of the link $e \in \mathbb{E}$
$C_{\text{src}}^e \in \mathbb{M}$	Source node of the link $e \in \mathbb{E}$
$C_{\text{dst}}^e \in \mathbb{M}$	Destination node of the link $e \in \mathbb{E}$
$C_{\text{node}}^e := C_{\text{src}}^e \cup C_{\text{dst}}^e$	Set of source and destination nodes of the link $e \in \mathbb{E}$.
$\bar{e} \in \mathbb{E}$	Opposite link of the link $e \in \mathbb{E}$
$\mathbb{M}_{\text{nb}}^m \subseteq \mathbb{M}$	Set of connected nodes (neighbors) of the node $m \in \mathbb{M}$

Table 2. VM model notation.

Parameter	Description
$\Psi_{\text{cpu}}^v \in \mathcal{R}_{\geq 0}$	Number of CPUs required by the VM $v \in \mathbb{V}$
$\Psi_{\text{ram}}^v \in \mathcal{R}_{\geq 0}$	Amount of RAM (in MB) required by the VM $v \in \mathbb{V}$
$\Psi_{\text{start}}^v \in [0, T]$	Arrival time (in second) of the VM $v \in \mathbb{V}$
$\Psi_{\text{dur}}^v \in [0, T]$	Duration time (in second) of the VM $v \in \mathbb{V}$
$\Psi_{\text{bw}}^{(v_1, v_2)} \in \mathcal{R}_{\geq 0}$	Bandwidth (in Mbps) required for the communication from VMs $v_1 \in \mathbb{V}$ to $v_2 \in \mathbb{V}$
$x_v \in \mathbb{S} \cup \emptyset$	The VM $v \in \mathbb{V}$ can be placed in a server or not (with a request rejection)
$y_{v_1}^{v_2} \in \mathbb{P} \cup \emptyset$	The network path can be selected for the communication from VMs $v_1 \in \mathbb{V}$ to $v_2 \in \mathbb{V}$ or not (with a request rejection)
$\vec{x}_v \in \{0, 1\}$	Indicates if the VM $v \in \mathbb{V}$ has to be migrated from a server to another one
$\vec{y}_{v_1}^{v_2} \in \{0, 1\}$	Indicates if the selected network path for the communication from VMs $v_1 \in \mathbb{V}$ to $v_2 \in \mathbb{V}$ is changed
$\delta_v \in \mathbb{G}$	Membership group of the VM $v \in \mathbb{V}$
$\tilde{\xi}_{\text{cpu}}^v \in [0, 1]$	Fraction of CPU guaranteed to the VM $v \in \mathbb{V}$

for VMs includes the computing/networking resources (CPU, RAM, link bandwidth) and power-aware classification (derived from QoS/SLA requirements).

We define $\mathbb{V}$ as the set of running VMs. The VM model is described in Table 2 (we use $\mathcal{R}_{\geq 0}$ for the set of non-negative real numbers). The parameter T refers to the simulation time period, and it will be described in detail in the next section. The VM placement is expressed with parameter $x_v \in \mathbb{S} \cup \emptyset$ that indicates the server where the VM $v \in \mathbb{V}$ is placed or an *empty* value ($\emptyset$)⁴ if the placement procedure is failed (i.e., rejected request). Instead, to represent

⁴ To be more precise, with $\emptyset$, we indicate that the variable is not assigned.

the selected network path for the communication between the VMs, we define the parameter $y_{v_1}^{v_2} \in \mathbb{P} \cup \emptyset$ that refers to the selected network path for the communication from VM v_1 to VM v_2 (with $v_1, v_2 \in \mathbb{V}$) or an empty value ($\emptyset$) if the network path selection is failed (rejected request). Instead, the binary variables $\vec{x}_v$ and $\vec{y}_{v_1}^{v_2}$ indicate if the VM $v \in \mathbb{V}$ has to be migrated from one server to another one and that the selected network path for the traffic communication between $v_1 \in \mathbb{V}$ to $v_2 \in \mathbb{V}$ is changed, respectively.

To represent the VM classification, we use the parameter $\delta_v \in \mathbb{G}$ that indicates the membership group. Each VM belongs to a one and only one group (see below). Instead, the parameter ξ_{cpu}^v represents the fraction of CPU guaranteed to the VMs by the group membership.

We assume a shared network storage because the VM live migration works only in this case. For this reason, the resources required by the VM do not include the storage requirements.

We consider the following power-aware labels:

- red** for very critical entities (e.g., they are currently processing medium/high loads, and/or there are tight QoS constraints on availability, processing, bandwidth, latency);
- yellow** for busy entities that are not operating close to their performance limit (e.g., they are currently processing low loads, and QoS constraints are largely met);
- green** for low-load entities (e.g., they are not currently processing anything or have a very low load, and there are negligible risks of SLA violations).

Based on the above classification, we define a set of groups $\mathbb{G}$ where each group g is described by the properties reported in Table 3. With the parameter ξ_{cpu}^g , we indicate the fraction of CPU guaranteed to the VMs. Based on this model, the parameter $\tilde{\xi}_{\text{cpu}}^v$ of Table 2 can be defined as

$$\tilde{\xi}_{\text{cpu}}^v := \xi_{\text{cpu}}^g \mid \delta_v = g \in \mathbb{G} \quad \forall v \in \mathbb{V} \quad (2)$$

Table 3. Group properties.

Parameter	Description
ξ_{name}^g	Name of the group $g \in \mathbb{G}$ (<i>red</i> , <i>yellow</i> or <i>green</i>)
$\xi_{\text{cpu}}^g \in [0, 1]$	Fraction of CPU guaranteed to the VMs belonging to the group $g \in \mathbb{G}$

3.3 Request Arrival and Departure Process

In this paper, we consider requests from tenants, which are generated randomly; where the arrival and departure process is assumed to follow an exponential distribution. The generic request r specified by a tenant consists of a new set of VMs $\mathbb{V}^{(r)}$. This set includes the same requirements defined in Table 2.

In addition, the request r arrives and has a duration in the interval $[0, T]$. For the sake of simplicity, we define the set of all requests as $\mathbb{R}$, with $r \in \mathbb{R}$.

The proposed request model differs from other works mainly in consideration of the *time* dimension. Unlike the work in [21], in this paper, we consider the time dimension in a continuous domain where the request can include VMs with different duration times.

3.4 Power Model

We consider a multi-component utilization-based power model, which defines the power consumption of a data center according to the resource utilization of servers and network devices. According to the typical non-linear behavior of power consumption for ICT devices [22,23], we define the power consumption of a device (server or network device) as a function of resource utilization U_m , which is the percentage of CPU utilization for servers and the fraction of active ports for network devices.

We identify three different power states: (i) *standby* – Γ_{stdby} , when the device is sleeping (equivalent to the ACPI S3 state); (ii) *idle* – Γ_{idle} , when the device is switched on but no operation is being carried out; and (iii) *active* – Γ_{act} , when the device is switched on and it performs operations.

Table 4. Power model notation.

Parameter	Description
Γ_{stdby}	Standby power state where the device is sleeping
Γ_{idle}	Idle power state where the device is switched on but no operation is begin carried out
Γ_{act}	Active power state where the device is switched on and performs operations
$\Gamma_{\text{ps}}^m \in \{\Gamma_{\text{stdby}}, \Gamma_{\text{idle}}, \Gamma_{\text{act}}\}$	Current power state configuration of the node $m \in \mathbb{M}$
$\Gamma_{\text{wu}}^m \in \{0, 1\}$	Indicates if the device $m \in \mathbb{M}$ has to wake-up changing the power state from standby to active/idle due to the last execution of the consolidation algorithm (see Sect. 4 for more details)
$\Gamma_{\text{slp}}^m \in \{0, 1\}$	Indicates if the device $m \in \mathbb{M}$ has to sleep changing the power state from active/idle to standby due to the last execution of the consolidation algorithm (see Sect. 4 for more details)
Φ_m^*	Standby power consumption of the device $m \in \mathbb{M}$
Φ_m^o	Idle power consumption of the device $m \in \mathbb{M}$
$\Phi_m^\bullet$	Power consumption of the device $m \in \mathbb{M}$ at full load
$\Phi_m(U_m, \Gamma_{\text{ps}}^m)$	Current power consumption of the device $m \in \mathbb{M}$ according to the utilization U_m and the power state Γ_{ps}^m

To represent the current power consumption, it is necessary to define the power state configuration of a device $m \in \mathbb{M}$ as $\Gamma_{\text{ps}}^m \in \{\Gamma_{\text{stdby}}, \Gamma_{\text{idle}}, \Gamma_{\text{act}}\}$. In addition, for the execution of the proposed consolidation algorithm described in Sect. 4, we introduce the binary variables Γ_{wu}^m and Γ_{slp}^m that indicate if the device has to *wake-up* (changing the power state from standby to active/idle) or *sleep* (changing the power state from active/idle to standby), respectively. Table 4 summarizes all the described parameters.

As a result, the total power consumption of device $m \in \mathbb{M}$ can be determined by a function $\Phi_m(U_m, \Gamma_{\text{ps}}^m)$:

$$\Phi_m(U_m, \Gamma_{\text{ps}}^m) = \begin{cases} \Phi_m^* & \text{if } \Gamma_{\text{ps}}^m = \Gamma_{\text{stdby}} \\ \Phi_m^\circ & \text{if } \Gamma_{\text{ps}}^m = \Gamma_{\text{idle}} \\ \Phi_m^\bullet + U_m & \text{if } \Gamma_{\text{ps}}^m = \Gamma_{\text{act}} \end{cases} \quad (3)$$

where Φ_m^* , Φ_m° are the standby and idle power consumption, respectively; while $\Phi_m^\bullet$ is the power consumption at full load.

4 Consolidation Algorithm

Roughly speaking, our target is the design of a consolidation strategy that:

- keeps *red* VMs on the smallest number of servers, while assuring enough resources are provisioned to meet their stringent QoS requirements;
- groups *yellow* VMs together on the smallest number of servers, even on the same servers as *red* VMs if enough resources are available;
- distributes *green* components among all servers, with no stringent constraints on grouping the largest number of them together, since servers hosting *green* components only will sleep;
- places VMs according to the total power consumption of active servers, network devices, and network links, in order to minimize the power consumption of the overall underlying data center infrastructure.

Despite the complexity and the dimensions of the problem and similar to works in the literature [4, 13, 20], we formulated the proposed consolidation strategy as a global optimization procedure with the aim to find a best possible trade-off between the (i) placement allocation and the (ii) network path selection.

The placement allocation identifies the group of servers where to allocate the VMs to satisfy the resource requirements described in Sect. 3.2. We want to guarantee the performance in terms of QoS or SLA and with the objective to minimize the total power consumption of servers. Instead, the selection of the network path is performed to satisfy the traffic requirements between the VMs in order to guarantee the necessary bandwidth and minimizing the total power consumption of network devices.

Due to the arrival and departure process of the requests and the complexity of the optimization procedure, it is difficult to find an optimal solution in a

reasonable period of time. For this reason, the optimization procedure is solved using various methods based on heuristics obtaining a sub-optimal solution as close as possible to the optimal one (see Sect. 4 for more details). It is important to note that there is not always a solution that satisfies the constraints. For this reason, the optimization procedure tries to place the largest number of VMs rejecting the other ones.

The strategy is shown in Algorithm 1. It starts with the deletion of the terminated VMs from the set $\mathbb{V}$ calling the `REMOVEDTERMINATEDVM` function (line 5). After this step, in line 5, it is called the `OPTIMIZATION` function that returns the updated set of the allocated VMs in the set $\mathbb{V}$. With the updated set of VMs, the consolidation algorithm updates the power state configuration of servers and network devices calling the `POWERSTATEUPDATE` function (line 7). Finally, it updates the network path set $\mathbb{P}$ calling the `NETWORKPATHUPDATE` function (line 8).

Algorithm 1. Consolidation

Require:

- 1: sets of allocated VMs $\mathbb{V}$ and requested VMs $\mathbb{V}^{(r)}$
- 2: sets of servers $\mathbb{S}$ and network machines $\mathbb{N}$
- 3: set of network paths $\mathbb{P}$

- 4: **function** `CONSOLIDATION`($\mathbb{V}, \mathbb{V}^{(r)}, \mathbb{S}, \mathbb{N}, \mathbb{P}$)
- 5: $\mathbb{V} \leftarrow \text{REMOVEDTERMINATEDVM}(\mathbb{V})$
- 6: $\mathbb{V} \leftarrow \text{OPTIMIZATION}(\mathbb{V}, \mathbb{V}^{(r)}, \mathbb{S}, \mathbb{N}, \mathbb{P})$
- 7: $(\mathbb{S}, \mathbb{N}) \leftarrow \text{POWERSTATEUPDATE}(\mathbb{V}, \mathbb{S}, \mathbb{N})$
- 8: $\mathbb{P} \leftarrow \text{NETWORKPATHUPDATE}(\mathbb{S}, \mathbb{N})$
- 9: **return** $\mathbb{V}, \mathbb{S}, \mathbb{N}, \mathbb{P}$

4.1 Remove Terminated VMs

This procedure removes the terminated VMs considering the arrival and duration time information. It returns an updated set of the allocated VMs $\mathbb{V}$ as shown in Algorithm 2.

4.2 Optimization Algorithm

We formulate the optimization algorithm as a function that returns the set of the allocated VMs. The problem aims to minimize the total power consumption of servers and network devices; while satisfying the resource constraints (in terms of CPUs, RAM, and network bandwidth) of the incoming requests as reported in (4–14).

(4) consists of the minimization of the power consumption of the servers and network devices with respect to the variables x_v (with $v \in \mathbb{V}^{(r)}$) and $y_{v_1}^{v_2}$ (with $v_1, v_2 \in \mathbb{V}^{(r)}$), respectively; while (5–14) represents the constraints.

Algorithm 2. Remove Terminated VMs**Require:**

```

1: set of allocated VMs  $\mathbb{V}$ 

2: function REMOVE_TERMINATED_VM( $\mathbb{V}$ )
3:    $\tilde{\mathbb{V}} \leftarrow \emptyset$ 
4:   for all  $v \in \mathbb{V}$  do
5:     if  $\Psi_{\text{start}}^v + \Psi_{\text{dur}}^v > t$  then
6:        $\tilde{\mathbb{V}} \leftarrow v$ 
7:   return  $\tilde{\mathbb{V}}$ 

```

Table 5. Optimization parameters.

Parameter	Description
$\vec{x}_*$	Maximum allowed percentage of VM migrations
$\vec{y}_*$	Maximum allowed percentage of re-routing of network communication between VMs
Γ_{chg}^*	Maximum allowed percentage of node power state reconfigurations

$$\min_{\substack{x_v | v \in \mathbb{V}^{(r)} \cup \mathbb{V} \\ y_{v_1}^{v_2} | v_1, v_2 \in \mathbb{V}^{(r)} \cup \mathbb{V}}} \sum_{s \in \mathbb{S}} \Phi_s(U_s, \Gamma_{\text{ps}}^s) + \sum_{n \in \mathbb{N}} \Phi_n(U_n, \Gamma_{\text{ps}}^n) \quad (4)$$

$$x_v \in \mathbb{S} \cup \emptyset \quad \forall v \in \mathbb{V}^{(r)} \quad (5)$$

$$y_{v_1}^{v_2} \in \mathbb{P} \cup \emptyset \quad \forall v_1, v_2 \in \mathbb{V}^{(r)} \quad (6)$$

$$\sum_{v \in \mathbb{V}^{(r)} | x_v = s} \Psi_{\text{cpu}}^v \times \tilde{\xi}_{\text{cpu}}^v \leq C_{\text{cpu}}^s - \sum_{v \in \mathbb{V} | x_v = s} \Psi_{\text{cpu}}^v \times \tilde{\xi}_{\text{cpu}}^v \quad \forall s \in \mathbb{S} \quad (7)$$

$$\sum_{v \in \mathbb{V}^{(r)} | x_v = s} \Psi_{\text{ram}}^v \leq C_{\text{ram}}^s - \sum_{v \in \mathbb{V} | x_v = s} \Psi_{\text{ram}}^v \quad \forall s \in \mathbb{S} \quad (8)$$

$$\sum_{v_1, v_2 \in \mathbb{V}^{(r)} | e \in y_{v_1}^{v_2}} \Psi_{\text{bw}}^{(v_1, v_2)} \leq C_{\text{bw}}^m - \sum_{v_1, v_2 \in \mathbb{V} | e \in y_{v_1}^{v_2}} \Psi_{\text{bw}}^{(v_1, v_2)} \quad \forall e \in \mathbb{E} \quad (9)$$

$$\frac{\sum_{v \in \mathbb{V}} \vec{x}_v}{|\mathbb{V}|} \leq \frac{\vec{x}_*}{100} \quad (10)$$

$$\frac{\sum_{v_1, v_2 \in \mathbb{V}} \vec{y}_{v_1}^{v_2}}{|\mathbb{P}|} \leq \frac{\vec{y}_*}{100} \quad (11)$$

$$\sum_{m \in \mathbb{M}} \Gamma_{\text{wu}}^m + \Gamma_{\text{slp}}^m \leq \Gamma_{\text{chg}}^* \quad (12)$$

$$U_s = \frac{\sum_{v \in \mathbb{V}^{(r)} \cup \mathbb{V} | x_v \in \mathbb{V}} \Psi_{\text{cpu}}^v \times \tilde{\xi}_{\text{cpu}}^v}{C_{\text{cpu}}^s} \quad \forall s \in \mathbb{S} \quad (13)$$

$$U_n = \frac{1}{\sigma_{n_{e \in \mathbb{E} | C_{\text{src}}^e = n}}} \times \sum \text{sgn}(C_u^e + C_u^{\bar{e}}) \quad \forall n \in \mathbb{N} \quad (14)$$

The first two Eqs. (5–6) represents the constraints related to the domains of the problems variables x_v (5), and $y_{v_1}^{v_2}$ (6). It is important to remember, that the optimization procedure tries to place the greatest possible number of VMs selecting the network path that guarantees the required bandwidth with the minimum impact on the global power consumption. However, it could happen that it is not possible to place some VMs. For this reason, the domains of the problem variables include the empty value ($\emptyset$), too.

The next two Eqs. (7–8) ensure that the VMs are placed on servers with sufficient available space for the requested resources (CPU and RAM, respectively). It is important to note that it is guaranteed only the CPU fraction $\tilde{\xi}_{\text{cpu}}^v$ for each VM $v \in \mathbb{V}$.

(9) guarantees that the total bandwidth to allocate is less or equal to the link capacity minus the bandwidth of the VMs already allocated.

Instead, (10–11) ensure a maximum percentage of migrations of VMs and re-routing of traffic between VMs, respectively. They use the parameters shown in Table 5. (12) guarantees a maximum percentage of power state transitions (wake-up and sleeping) for all the nodes in the data center (servers and network devices).

Finally, the last two Eqs. (13–14) are used to compute the utilization level of the server $s \in \mathbb{S}$ as a fraction of the CPU utilized by the VMs, and of the network device $n \in \mathbb{N}$ as a fraction of the utilized ports, respectively.

4.3 Search Methods and Heuristics

To implement the optimization algorithm we used the Optaplanner constraint satisfaction solver [24]. This framework supports three families of optimization algorithms: Exhaustive Search (ES), Construction Heuristics (CH) and Meta-Heuristics (MH).

The algorithms of the ES family always find the global optimum and recognize it too. However, they do not scale (not even beyond small data sets) and they are mostly useless. Indeed, they suffer scalability issues regarding memory and performance. In our simulation, we consider the MH family (in combination with CH ones to initialize) which are the recommended choice to deliver a good result in a reasonable period of time.

In more details, the CH build a pretty good initial solution in a finite length of time. Its solution is not always feasible, but it finds it fast so MH can finish the job. The heuristic types considered in this work are summarized in Table 6.

Table 6. Meta-Heuristics algorithm types.

Name	Description
Hill Climbing (HC)	Select the change respect to the initial configuration that performs the minimum cost. It can easily get stuck in a local optimum. This happens when it reaches a solution for which all the moves increase the cost [25]
Tabu Search (TS)	Maintains a tabu list to avoid local optimum. The tabu list holds recently used objects that are taboo to use for now. Changes that involve an object in the tabu list are not accepted [26, 27]
Simulated Annealing (SA)	Evaluates only a few changes per step. The first accepted change is the winning step. A change is accepted if does not increase the cost [28]
Late Acceptance (LA)	Evaluates only a few changes per step. A change is accepted if it does not increase the cost, or if it leads to a cost that is at least the minimum one of a fixed number of steps ago [29]

Start from an initial solution evolving into a mostly better solution

4.4 Power State Update

After the execution of the optimization problem, it is necessary to update the power state configuration of servers and network devices in the data center in order to put in standby states the inactive devices. As shown in Algorithm 3, this function is separated into two parts: (i) server update and (ii) network update. The former part is realized calling the Function `POWERSTATESERVERUPDATE` (lines 7–15) that updates the power state of the servers in the following ways:

- sleep** transition for servers in the active state without VMs belonging to the *red* or *yellow* groups (lines 12–14);
- wake-up** transition for servers in the standby state with at least one VMs belonging to the *red* or *yellow* groups (lines 12–14).

As consequence of this step, it is possible to update the power state configuration of the network devices, too. To be more precise, it is called the Function `POWERSTATENETWORKUPDATE` (lines 16–24) that updates the network devices in the following ways:

- sleep** transition for network devices with traffic utilization level equal to 0 for all the connected link interfaces (lines 9–11);
- wake-up** transition for network devices with positive traffic utilization level for at least one connected link interface (lines 21–23).

It is important to remember that the VMs belonging the green group do not perform any elaboration, so the traffic utilization level of a link C_u^e can be positive only for the contribution of the VMs belonging to the *red* or *yellow* groups.

Algorithm 3. Power State Update**Require:**

```

1: set of allocated VMs  $\mathbb{V}$ 
2: sets of servers  $\mathbb{S}$  and network machines  $\mathbb{N}$ 

3: function POWERSTATEUPDATE( $\mathbb{V}, \mathbb{S}, \mathbb{N}$ )
4:    $\mathbb{S} \leftarrow \text{POWERSTATESERVERUPDATE}(\mathbb{V}, \mathbb{S})$ 
5:    $\mathbb{N} \leftarrow \text{POWERSTATENETWORKUPDATE}(\mathbb{N})$ 
6:   return  $\mathbb{S}, \mathbb{N}$ 

7: function POWERSTATESERVERUPDATE( $\mathbb{V}, \mathbb{S}$ )
8:   for all  $s \in \mathbb{S}$  do
9:     if  $\Gamma_{ps}^s = \Gamma_{act}$  then
10:      if  $\neg \exists v \in \mathbb{V} | x_v = s \wedge \delta_v = g \wedge \xi_{name}^g = red$  then
11:         $\Gamma_{ps}^s \leftarrow \Gamma_{stdby}$ 
12:      else if  $\Gamma_{ps}^s = \Gamma_{stdby}$  then
13:        if  $\exists v \in \mathbb{V} | x_v = s \wedge \delta_v = g \wedge \xi_{name}^g \neq green$  then
14:           $\Gamma_{ps}^s \leftarrow \Gamma_{act}$ 
15:   return  $\mathbb{S}$ 

16: function POWERSTATENETWORKUPDATE( $\mathbb{N}$ )
17:   for all  $n \in \mathbb{N}$  do
18:     if  $\Gamma_{ps}^n = \Gamma_{act}$  then
19:       if  $\neg \exists e \in \mathbb{E} | n \in C_{node}^e \wedge C_u^e > 0$  then
20:          $\Gamma_{ps}^n \leftarrow \Gamma_{stdby}$ 
21:       else if  $\Gamma_{ps}^n = \Gamma_{stdby}$  then
22:         if  $\exists e \in \mathbb{E} | n \in C_{node}^e \wedge C_u^e > 0$  then
23:            $\Gamma_{ps}^n \leftarrow \Gamma_{act}$ 
24:   return  $\mathbb{N}$ 

```

4.5 Network Path Update

The change of the power state configuration of servers and network devices makes necessary to update the network paths composing the set $\mathbb{P}$ in order to remove the paths with nodes in standby mode. To implement this procedure in order to perform a fast execution, we use a reference set $\mathbb{P}$ computed at the beginning (before to start the simulation, when all the devices are in standby mode). Each time this function is called, the updated set is computed considering only the paths with at least one node in the active state (Algorithm 4).

5 Performance Evaluation

In this section, we validate the effectiveness of the proposed algorithm using simulation and numerical results. We show the efficiency of the consolidation algorithm based on different heuristics to process the incoming VM allocation requests, and the impact of the migration regarding power, energy-saving, and resource overhead.

Algorithm 4. Network Path Update**Require:**

- 1: reference set of network paths $\tilde{\mathbb{P}}$
- 2: sets of servers $\mathbb{S}$ and network machines $\mathbb{N}$

- 3: **function** NETWORKPATHUPDATE($\mathbb{S}, \mathbb{N}$)
- 4: $\mathbb{P} \leftarrow \emptyset$
- 5: **for all** $p \in \tilde{\mathbb{P}}$ **do**
- 6: **if** $\exists e \in p \mid m \in C_{\text{node}}^e \wedge \Gamma_{\text{ps}}^m \neq \Gamma_{\text{stdby}}$ **then**
- 7: $\mathbb{P} \leftarrow p$
- 8: **return** $\mathbb{P}$

5.1 Simulation Settings

As in [21], we consider a multi-rooted hierarchical topology that connects the switches in three layers (Core, Aggregation, and Edge). A group of servers in a rack is connected to an edge layer switch, which in turn is connected to aggregation switches at the next layer. Each aggregation switch is connected to a group of core switches in the next layer which in turn are attached to some front-end servers for connecting to the external clients. In particular, we choose a DCN (Data-Center Network) with fat-three topology [30,31].

In our simulation, we choose $k = 8$ with a total number of switches and servers equal to 80 and 128, respectively and with 1 Gbps capacity set for the links. We assign for each server the same amount of resources: 128 GB of RAM and 32 CPUs per server.

For the power values, we follow the references [16,21,23,32,33], where the edge, aggregation, and core switches have: (i) the idle power usage of 76.40 W, 133.50 W, and 555 W; and (ii) the maximum power usage at full load of 102 W, 175 W, and 656 W; respectively. Instead, for the standby power usage, considering that most of the components are disabled and only the RAM is on, the power consumption is usually less than 5 W for all the switches.

As described in [21], it has been observed that a server with two cores consumes 170 W and 220 W in the idle and fully utilized states, respectively. Since these power values have a direct relationship with the number of its cores [23], we scale up the server's power based on its resources considered in our simulations. Accordingly, the power usage of a server with 32 CPUs in idle and fully utilized states is 2720 W and 3520 W, respectively. Similar to the switch case, also for the server the standby power is less than 5 W.

For what concerns the set of groups $\mathbb{G}$ and its relative properties ξ_{cpu}^g we consider the following values. For the VMs of the *red* group, it is guaranteed the total amount of CPU, and they have to be placed with a maximum distance of 1 hop. Instead, for the VMs of the *yellow* group, it is guaranteed a reduced fraction of CPU: 0.2 and they have to be placed with a maximum distance of 5 hops. Finally, for the VMs of the *green* group, it is not guaranteed a fraction of CPU, and they can be placed without constraints about the distance.

5.2 Metrics

To make an exhaustive analysis of the performance evaluation of the proposed consolidation algorithm for all the heuristics described Sect. 4.3, we take into account the following metrics: (i) *energy savings*, computed considering the power consumption of the Data center without energy savings mechanisms enabled (e.g. frequency/voltage scaling and sleep states) but still proportional to the resource utilization; (ii) *acceptance rate*, in terms of the average percentage of the accepted VM requests per seconds; and (iii) *migration rate*, in terms of the average percentage of the performed VM migrations per seconds.

5.3 Scenario

We choose a simulation time of $T = 24$ h. Unlike other works [21], in this paper, the simulation time domain is continuous. Hence, a new request can arrive at any time between 0 and 24 h. In order to compare the performance of the proposed consolidation algorithm with the different mentioned heuristics, we consider the following scenario. The mean arrival rate of the incoming requests follows the profile shown in Fig. 1. There are three areas characterized by three different levels of average incoming requests per second. The lowest values (≈ 400 requests per second) occur in the first part from 0:00 to 8:00. Instead, in the second part of the trace, the number of requests per seconds is approximately equal to 800. Finally, in the third part, the number of requests per second reaches the maximum value equal to 1200.

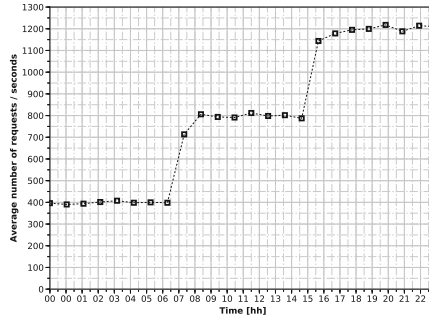


Fig. 1. Profile of the average number of requests per second.

In the literature, data center traffic characteristics at the packet level between switches have been studied, and traffic distribution such as exponential, Weibull, and log-normal have been reported [34, 35]. We choose the Log-normal distribution for the number of VMs for request. We vary the mean request duration in the range of $[1, 10]$ seconds following a Log-normal distribution with mean equal to 5 s. All the parameters and distributions used in our simulations are reported

in Table 7. With $\Psi_{\text{cpu}}^{(\mu)}$, $\Psi_{\text{ram}}^{(\mu)}$, and $\Psi_{\text{bw}}^{(\mu)}$, we refer to the mean of the distributions of the resources required by the VMs (CPUs, RAM, and bandwidth, respectively). Finally, the upper bounds for the migrations of VMs, the re-routing of traffic between VMs and the power state transitions we choose 25%, 15%, and 25%, respectively.

Table 7. Configuration parameters used in the simulation.

Parameter	Value		
T	24 h		
Parameter	Value	Distribution	Range
μ_{vm}^r	20 VMs	Log-normal	[0, 40] VMs
μ_{dur}^r	5 s	Log-normal	[1, 10] seconds
$\Psi_{\text{cpu}}^{(\mu)}$	3 CPUs	Log-normal	[0, 10] CPUs
$\Psi_{\text{ram}}^{(\mu)}$	20 GB	Log-normal	[1, 10] GB
$\Psi_{\text{bw}}^{(\mu)}$	100 Mbps	Log-normal	[0, 200] Mbps
$\vec{x}_*$	25%		
$\vec{y}_*$	15%		
Γ_{chg}^*	25%		

Instead, the VM group membership is assigned based on the Uniform distribution probability (*red*: 20%, *yellow*: 50%, and *green*: 30%).

5.4 Numerical Results

We wrote an event simulator in Java using the Optaplanner constraint satisfaction solver [24] to evaluate the proposed algorithm. We validated the effectiveness of our energy-aware consolidation algorithm through a comparison of the different heuristics with the results obtained maximizing only the performance.

We carry out the experiments for a sufficient number of times to get small errors representing the 95% Confidence Interval (CIs). The measured CIs have not been reproduced in the graphs since all of them are small and just clutter the figures.

Figure 2 displays the average percentage of the energy savings. Considering that the proposed algorithm has the objective to minimize the power consumption limiting the degradation of the performance, the results show a maximum energy saving of more than the 50% with the LA heuristics. In general, there is a significant saving of power consumption when the traffic load is low. Compared to the other heuristics, HC obtains worst results because that can easily get stuck in local optima. For this reason, to avoid the problem of local minima, the other heuristics implement more advanced solutions. In particular, with a low number of requests, in the LA case the energy savings is stable with values

higher than 20%. It is important to note how the trend, varying the request rate, is much more variable for the TS, SA and LA heuristics compared to the HC one. Indeed, these heuristics provide mechanisms to explore the search space in a better way. However, even with these methods, it is difficult to find a global minimum.

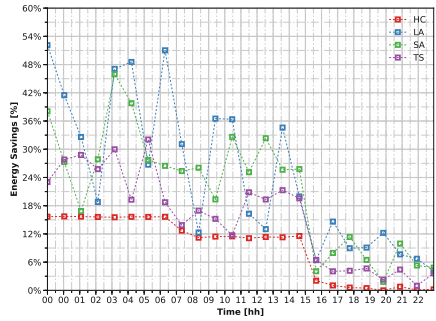


Fig. 2. Average percentage of the energy savings.

The acceptance rate is shown in Fig. 3. The difference between the TS, SA and LA heuristics are not as significant as in the case of the energy savings. The trend of the acceptance rate varies widely for the HC, too. Obviously, the problem of the local minima impacts also the acceptance rate. Indeed, for high values of incoming requests, HC presents an acceptance rate below 74% compared to the 86% of the TS case. An interesting point is the fact that TS and SA behave relatively better than LA (especially with a high number of requests). Most likely, the advanced solutions introduced by LA allow obtaining better results from the point of view of optimization (less power consumption) at the cost of a worse allocation of the resources.

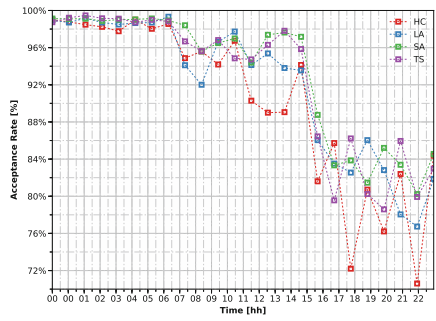


Fig. 3. Average percentage of the accepted requests per second.

The migration rate is shown in Fig. 4. The results are qualitatively significant only with a high number of incoming requests per second (the last part of the trace). With low/medium values of request rate, the differences between the heuristics are practically near to zero. Instead, with high values, SA performs better with a migration rate below 10% in some cases. As described for the acceptance rate, also for this metric the LA behaves relatively worse than TS and SA.

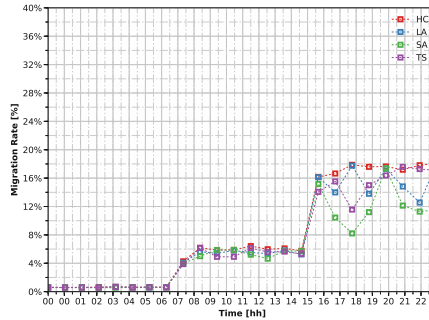


Fig. 4. Average percentage of the migrations performed per second.

6 Conclusions

In this work, we have proposed a new consolidation mechanism to reduce the power consumption in data centers. We have extended the state-of-the-art optimization problem, taking into account the presence of VMs with different roles in the overall service/application framework. The proposed approach allows to obtain high levels of energy saving, put to sleep servers that only host redundant software components.

Our consolidation strategy improves the efficiency of the infrastructure, with limited implications on the acceptance rate of auxiliary service requests. The next steps will be: (i) more rigorous integration between the server and network models, to obtain better power savings; (ii) design of a more advanced power model for network devices, considering the energy consumption for different operating rates; and (iii) detailed study and analysis of further heuristics, to find optimal allocations even with large data centers.

Acknowledgment. This work was partially supported by the European Commission under the projects ARCADIA (grant no. 607881) and INPUT (grant no. 644672).

References

1. Hammadi, A., Mhamdi, L.: A survey on architectures and energy efficiency in data center networks. *Comput. Commun.* **40**, 1–21 (2014)
2. Advanced configuration and power interface specification (2015). <http://www.uefi.org/sites/default/files/resources/ACPI.6.0.pdf>
3. Barroso, L.A., Hlzl, U.: The case for energy-proportional computing. *Computer* **40**(12), 33–37 (2007)
4. Stage, A., Setzer, T.: Network-aware migration control and scheduling of differentiated virtual machine workloads. In: *Proceedings of the 2009 ICSE Workshop on Software Engineering Challenges of Cloud Computing (CLOUD)*, Vancouver, Canada, 23 May 2009, pp. 9–14 (2009)
5. Voorsluys, W., Broberg, J., Venugopal, S., Buyya, R.: Cost of virtual machine live migration in clouds: a performance evaluation. In: Jaatun, M.G., Zhao, G., Rong, C. (eds.) *CloudCom 2009*. LNCS, vol. 5931, pp. 254–265. Springer, Heidelberg (2009). doi:[10.1007/978-3-642-10665-1_23](https://doi.org/10.1007/978-3-642-10665-1_23)
6. Herbst, N.R., Kounev, S., Reussner, R.: Elasticity in cloud computing: what it is, and what it is not. In: *Proceedings of the 10th International Conference on Autonomic Computing (ICAC)*, San Jose, CA, USA, 24–28 June 2013, pp. 23–27 (2013)
7. Kliazovich, D., Bouvry, P.: DENS: data center energy-efficient network-aware scheduling. In: *IEEE/ACM International Conference on Green Computer and Communication & IEEE/ACM International Conference on Cyber, Physical and Social Computing*, Hangzhou, China, 18–20 December 2010, pp. 69–75 (2010)
8. Topology and orchestration specification for cloud applications. OASIS Standard, November 2013, version 1.0. <http://docs.oasis-open.org/tosca/TOSCA/v1.0/os/TOSCA-v1.0-os.pdf>
9. Wettinger, J., Breitenbücher, U., Leymann, F.: Standards-based DevOps automation and integration using TOSCA. In: *IEEE/ACM 7th International Conference on Utility and Cloud Computing (UCC)*, London, UK, 8–11 December 2014, pp. 59–68 (2014)
10. Balalaie, A., Heydarnoori, A., Jamshidi, P.: Microservices architecture enables devops: migration to a cloud-native architecture. *IEEE Softw.* **33**(3), 42–52 (2016)
11. Dragoni, N., Giallorenzo, S., Lafuente, A.L., Mazzara, M., Montesi, F., Mustafin, R., Safina, L.: Microservices: yesterday, today, and tomorrow. Cornell University Library, [arXiv:1606.04036v1](https://arxiv.org/abs/1606.04036v1), June 2016
12. Srikantaiah, S., Kansal, A., Zhao, F.: Energy aware consolidation for cloud computing. In: *Proceedings of the 2008 Conference on Power Aware Computer and Systems (HotPower 2008)*, San Diego, CA, USA, 7 December 2008 (2008)
13. Mishra, M., Sahoo, A.: On theory of VM placement: anomalies in existing methodologies and their mitigation using a novel vector based approach. In: *IEEE International Conference on Cloud Computing (CLOUD)*, Bombay, Mumbai, India, 4–9 July 2011, pp. 275–282 (2011)
14. Geronimo, G.A., Werner, J., Westphall, C.B., Westphall, C.M., Defenti, L.: Provisioning and resource allocation for green clouds. In: *The 12th International Conference on Network (ICN)*, Seville, Spain, 27 January–1 February 2013
15. Beloglazov, A., Buyya, R.: OpenStack Neat: a framework for dynamic and energy-efficient consolidation of virtual machines in OpenStack clouds. *Conc. Comput. Pract. Exp.* **27**(5), 1310–1333 (2015)

16. Heller, B., Seetharaman, S., Mahadevan, P., Yiakoumis, Y., Sharma, P., Banerjee, S., McKeown, N.: Elastictree: saving energy in data center networks. In: Proceedings of the 7th USENIX Conference on Network System Design and Implementation, ser. NSDI, p. 17. USENIX Association, Berkeley (2010)
17. Shirayanagi, H., Yamada, H., Kono, K.: Honeyguide: a VM migration-aware network topology for saving energy consumption in data center networks. In: IEEE Symposium on Computers and Communications (ISCC), Cappadocia, Turkey, 1–4 July 2012, pp. 460–467 (2012)
18. Wang, L., Zhang, F., Vasilakos, A.V., Hou, C., Liu, Z.: Joint virtual machine assignment and traffic engineering for green data center networks. *ACM SIGMETRICS Perf. Eval. Rev.* **41**(3), 107–112 (2013)
19. Zhang, Y., Ansari, N.: HERO: hierarchical energy opt. for data center networks. *IEEE Syst. J.* **9**(2), 406–415 (2015)
20. Fang, W., Liang, X., Li, S., Chiaraviglio, L., Xiong, N.: VMPlanner: optimizing virtual machine placement and traffic flow routing to reduce network power costs in cloud data centers. *Comput. Netw.* **57**(1), 179–196 (2013)
21. Dalvandi, A., Gurusamy, M., Chua, K.C.: Time-aware vmflow placement, routing, and migration for power efficiency in data centers. *IEEE Trans. Netw. Serv. Man.* **12**(3), 349–362 (2015)
22. Padala, P., Shin, K.G., Zhu, X., Uysal, M., Wang, Z., Singhal, S., Merchant, A., Salem, K.: Adaptive control of virtualized resources in utility computing environments. In: Proceedings of the 2Nd ACM SIGOPS/EuroSys European Conference on Computer Systems: ser. EuroSys 2007, pp. 289–302. ACM, New York (2007)
23. Mahadevan, P., Sharma, P., Banerjee, S., Ranganathan, P.: Energy aware network operations. In: Proceedings of the 28th IEEE International Conference on Computer Communication Workshops, ser. INFOCOM 2009, pp. 25–30. IEEE Press, Piscataway (2009)
24. Optaplanner - constraint satisfaction solver (java, open source) (2016). <http://www.optaplanner.org>
25. Russell, S.J., Norvig, P., Intelligence, A.: A Modern Approach, 2nd edn. Pearson Education, Upper Saddle River (2003)
26. Glover, F.: Tabu search part I. *ORSA J. Comput.* **1**(3), 190–206 (1989)
27. Glover, F.: Tabu searchpart II. *ORSA J. Comput.* **2**(1), 4–32 (1990)
28. Khachatryan, A., Semenovskaya, S., Vainshtein, B.: The thermodynamic approach to the structure analysis of crystals. *Acta Crystallogr. Sect. A* **37**(5), 742–754 (1981)
29. Burke, E.K., Bykov, Y.: The late acceptance hill-climbing heuristic. *Europ. J. Opt. Res.* **258**(1), 70–78 (2017)
30. Al-Fares, M., Loukissas, A., Vahdat, A.: A scalable, commodity data center network architecture. *SIGCOMM Comput. Commun. Rev.* **38**(4), 63–74 (2008)
31. Niranjana Mysore, R., Pamboris, A., Farrington, N., Huang, N., Miri, P., Radhakrishnan, S., Subramanya, V., Vahdat, A.: Portland: a scalable fault-tolerant layer 2 data center network fabric. *SIGCOMM Comput. Commun. Rev.* **39**(4), 39–50 (2009)
32. Mahadevan, P., Sharma, P., Banerjee, S., Ranganathan, P.: A power benchmarking framework for network devices. In: Fratta, L., Schulzrinne, H., Takahashi, Y., Spaniol, O. (eds.) NETWORKING 2009. LNCS, vol. 5550, pp. 795–808. Springer, Heidelberg (2009). doi:[10.1007/978-3-642-01399-7_62](https://doi.org/10.1007/978-3-642-01399-7_62)

33. Nam, T.M., Thanh, N.H., Thu, N.Q., Hieu, H.T., Covaci, S.: Energy-aware routing based on power profile of devices in data center networks using SDN. In: 2015 12th International Conference on Electrical Engineering/Electronics Computer, Telecommunication and nformation Technology (ECTI-CON), pp. 1–6, June 2015
34. Benson, T., Anand, A., Akella, A., Zhang, M.: Understanding data center traffic characteristics. *SIGCOMM Comput. Commun. Rev.* **40**(1), 92–99 (2010)
35. Benson, T., Akella, A., Maltz, D.A.: Network traffic characteristics of data centers in the wild. In: Proceedings of the 10th ACM SIGCOMM Conference on International Measures, ser. IMC 2010, pp. 267–280. ACM, New York (2010)